

Laboratorio 3

ICN-292 — SISTEMAS DE INFORMACIÓN PARA LA GESTIÓN

Charles Piket Silva

Ingeniería Civil Industrial, 202360512-3, cpiket@usm.cl

Repositorio GitHub: <https://github.com/Cpikets/ICN292-Lab3-Piket-Charles.git>

Resumen ejecutivo

Se desarrolló en n8n un flujo para gestionar las devoluciones de AndesHogar SpA. El proceso recibe solicitudes mediante un Webhook, aplica las reglas de triage en el orden definido, consulta el valor de la UF, registra el resultado, envía una notificación al cliente y responde a la llamada HTTP. Para la semilla personal $S = 794$, los parámetros utilizados fueron $U = \$74,000$ y $D = 21$ días.

En la prueba base se procesaron las 15 solicitudes entregadas. Cinco fueron aprobadas automáticamente, seis quedaron en revisión y cuatro fueron rechazadas. Ninguna fue clasificada como dato inválido. El monto total procesado fue de \$1.251.820 y la tasa de aprobación automática fue de 33,33 %.

También se implementó un workflow programado que consolida diariamente el registro de devoluciones y genera un correo con solicitudes y montos por ruta, monto total del día y tasa de aprobación automática. De forma adicional se configuró un Error Workflow para notificar fallas. Las pruebas de casos borde confirmaron que $\text{monto} = U$ y $\text{dias_desde_compra} = D$ permanecen dentro de lo aceptado. También se comprobó que un SKU inexistente puede ser aprobado porque el flujo actual no consulta el catálogo, y que un monto no numérico puede hacer fallar el nodo Switch al intentar convertirlo.

Como MVP, la solución permite automatizar el triage y mantener trazabilidad, pero no se recomienda convertirla todavía en el sistema definitivo. Antes se deberían incorporar validación de esquema para entradas no numéricas, validación del SKU contra el catálogo, evidencias completas de manejo de errores y controles operacionales para dependencias externas como Gmail y la consulta de UF. Con esas correcciones, el flujo actual puede servir como base para una versión definitiva.

1. Parámetros personalizados y reglas de negocio

Para $S = 794$, los parámetros se calculan de la siguiente forma:

$$U = 30,000 + 1,000 (794 \bmod 50) = 74,000,$$

$$D = 7 + 7 (794 \bmod 4) = 21 \text{ días.}$$

Las reglas se aplicaron en el orden indicado en el enunciado. La primera condición que se cumple determina la ruta y el motivo asociado a la solicitud:

1. DATOS_INVALIDOS: id_solicitud vacío o $\text{monto} \leq 0$.
2. RECHAZO: $\text{dias_desde_compra} > 21$ o $\text{estado_producto} = \text{danado_por_uso}$.
3. REVISION: $\text{monto} > 74000$ o $\text{estado_producto} = \text{con_fallas}$.
4. APROBACION: cualquier solicitud que no cumpla una regla anterior.

Las comparaciones de monto y plazo son estrictas. Un monto igual a U no entra a revisión por monto y una solicitud con exactamente D días no se rechaza por plazo. Esta condición se verificó además con pruebas específicas descritas en la Parte C.

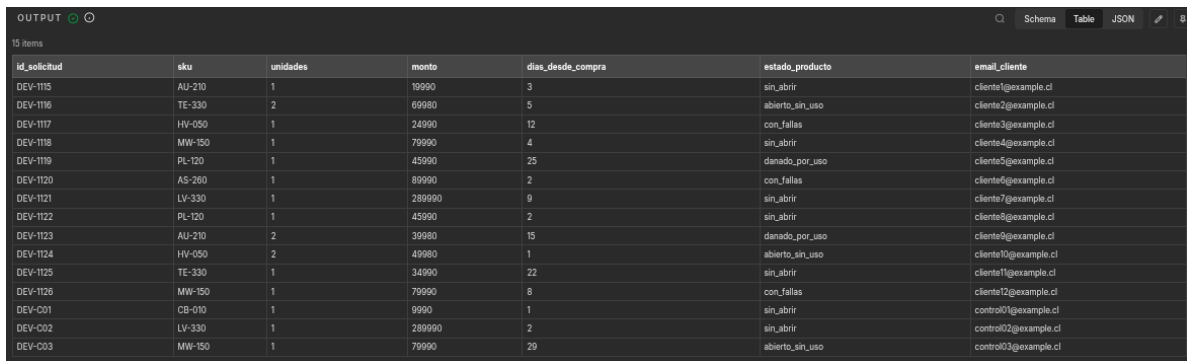
2. Parte A — Workflow principal de triage

2.1. Entrada y recepción de solicitudes

El workflow principal recibe las solicitudes mediante un Webhook configurado con método POST y path *devoluciones*. El lote se recibe dentro de *body* y el nodo *Normalizar entrada* separa el arreglo para continuar con un ítem por solicitud.

n8n entrega una *Test URL* y una *Production URL*. De acuerdo con la documentación oficial del nodo Webhook, la Test URL se registra al escuchar o ejecutar una prueba y permite ver los datos directamente en el editor; la Production URL queda disponible cuando el workflow está publicado y sus ejecuciones se revisan desde el historial. La evidencia final del lote base corresponde a una ejecución en modo de producción. La referencia utilizada fue <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook/>.

El lote base contiene 15 solicitudes. La Figura 1 muestra el conjunto utilizado en la prueba.



id_solicitud	sku	unidades	monto	dias_desde_compra	estado_producto	email_cliente
DEV-1115	AU-210	1	19990	3	sin_abrir	cliente1@example.cl
DEV-1116	TE-330	2	69980	5	abierto_sin_uso	cliente2@example.cl
DEV-1117	HV-050	1	24990	12	con_fallas	cliente3@example.cl
DEV-1118	MW-150	1	79990	4	sin_abrir	cliente4@example.cl
DEV-1119	PL-120	1	45990	25	danado_por_uso	cliente5@example.cl
DEV-1120	AS-260	1	89990	2	con_fallas	cliente6@example.cl
DEV-1121	LV-330	1	289990	9	sin_abrir	cliente7@example.cl
DEV-1122	PL-120	1	45990	2	sin_abrir	cliente8@example.cl
DEV-1123	AU-210	2	39980	15	danado_por_uso	cliente9@example.cl
DEV-1124	HV-050	2	49980	1	abierto_sin_uso	cliente10@example.cl
DEV-1125	TE-330	1	34990	22	sin_abrir	cliente11@example.cl
DEV-1126	MW-150	1	79990	8	con_fallas	cliente12@example.cl
DEV-001	CB-010	1	9990	1	sin_abrir	control01@example.cl
DEV-002	LV-330	1	289990	2	sin_abrir	control02@example.cl
DEV-003	MW-150	1	79990	29	abierto_sin_uso	control03@example.cl

Figura 1: Lote de 15 solicitudes utilizado como entrada del workflow principal.

2.2. Clasificación de las solicitudes

El nodo Switch está configurado en modo *Rules*. La validación de tipos es estricta y las condiciones numéricas convierten los textos recibidos mediante *toNumber()* antes de comparar. Las seis reglas se mantienen separadas para conservar un motivo específico por causal:

Orden	Condición	Salida
1	<code>id_solicitud</code> vacío	INVALIDO_CAMPO
2	<code>monto.toNumber() ≤ 0</code>	INVALIDO_MONTO
3	<code>dias_desde_compra.toNumber() > 21</code>	RECHAZO_PLAZO
4	<code>estado_producto = danado_por_uso</code>	RECHAZO_ESTADO
5	<code>monto.toNumber() > 74000</code>	REVISION_MONTO
6	<code>estado_producto = con_fallas</code>	REVISION_FALLA
—	Ninguna regla anterior se cumple	APROBACION (Fallback)

Tabla 1: Reglas configuradas en el nodo Switch.

El *Fallback Output* se utiliza como rama de aprobación. La configuración no envía un mismo ítem a todas las salidas coincidentes, de modo que se conserva la precedencia de la política. Las dos salidas de datos inválidos se reúnen en *Unir invalidos*; las dos de rechazo, en *Unir rechazo*; y las dos de revisión, en *Unir revision*. Después se aplica un nodo Edit Fields por ruta.

Los Edit Fields escriben *ruta*, *motivo*, *U* y *D*, además de conservar los datos de la solicitud. Para DATOS_INVALIDOS el motivo distingue entre *id_solicitud invalida* y *Monto invalido*; para RECHAZO distingue entre *Plazo maximo excedido* y *Producto danado por uso*; para REVISION dis-

tingue entre Monto sobre el umbral y Producto con fallas; y para APROBACION utiliza Cumple condiciones de aprobacion. Las cuatro rutas se reúnen luego en *Decision consolidada*.

La Figura 2 muestra la ejecución completa del lote base. Se reciben 15 solicitudes y se obtienen 4 rechazos, 6 revisiones y 5 aprobaciones. Las capturas de configuración detallada del Switch se dejaron en el Anexo A.

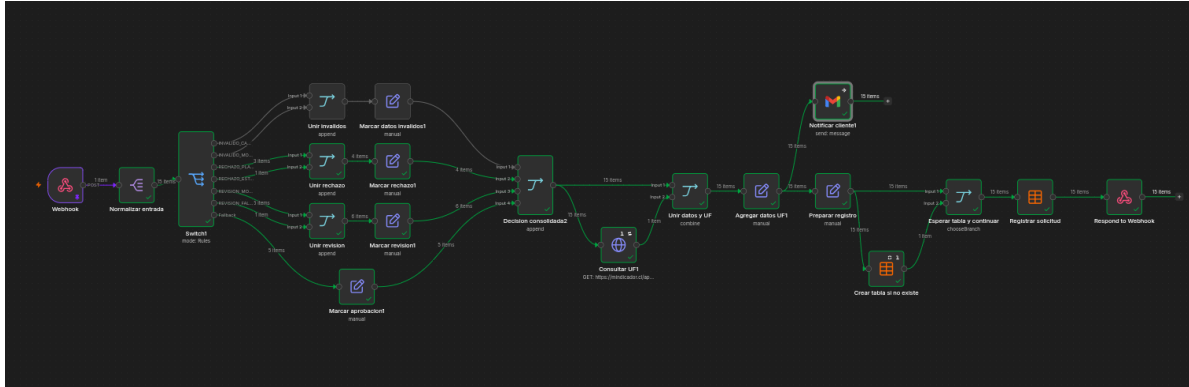


Figura 2: Ejecución final del workflow principal de triage de devoluciones.

2.3. Consulta de UF

Después de clasificar las solicitudes, el nodo *Consultar UF1* realiza una solicitud a <https://mindicador.cl/api/uf>. La consulta se ejecuta una sola vez y se combina con todas las solicitudes mediante el nodo *Unir datos y UF*.

El campo consumido de la respuesta es `serie[0].valor`. Ese valor se guarda como `valor_uf` y el monto equivalente se calcula como:

$$\text{monto_uf} = \frac{\text{monto}}{\text{serie}[0].\text{valor}},$$

redondeado a dos decimales. El nodo HTTP tiene activado *Retry On Fail*, con un máximo de tres intentos y una espera de 1000 ms entre intentos.

2.4. Registro, notificación y respuesta

Después de incorporar el valor de la UF, el flujo se divide en dos ramas. Una envía una notificación individual mediante Gmail al campo `email_cliente`; el mensaje informa identificador, ruta, motivo, monto y monto aproximado en UF. La otra prepara el registro, comprueba que exista la tabla `registro_devoluciones`, guarda los datos y termina en *Respond to Webhook*.

El registro conserva `fecha_registro`, `id_solicitud`, SKU, monto, días desde la compra, estado, ruta, motivo, *U*, *D*, valor de la UF, monto en UF y correo del cliente. La fecha se obtiene al momento de la ejecución y la tabla se crea automáticamente si no existe.

La operación de registro es *Upsert* utilizando `id_solicitud` como condición. Esto evita duplicar las mismas solicitudes al repetir una prueba: un identificador existente se actualiza y uno nuevo se agrega. El nodo *Respond to Webhook* devuelve todos los ítems recibidos desde el registro.

La Figura 3 muestra el registro obtenido después de procesar el lote completo.

fecha_registro	id_solicitud	sku	monto	dias_desde_compra	estado_producto	ruta	motivo	U	D	valor_uf	monto_uf	email_cliente	id	createdAt	updatedAt
2026-09-17T16:41:54.856-03:00	DEV-1119	PL-120	45990	25	danado_por_uso	RECHAZO	Plazo maximo excedido	74000	21	40950.9	1.12	cliente5@example.cl	1	2026-09-17T19:19:47.435Z	2026-09-17T19:41:54.885Z
2026-09-17T16:41:54.862-03:00	DEV-1125	TE-330	34990	22	sin_abrir	RECHAZO	Plazo maximo excedido	74000	21	40950.9	0.95	cliente11@example.cl	2	2026-09-17T19:19:47.493Z	2026-09-17T19:41:55.044Z
2026-09-17T16:41:54.876-03:00	DEV-C03	MW-150	79990	29	abierto_sin_uso	RECHAZO	Plazo maximo excedido	74000	21	40950.9	1.95	control03@example.cl	3	2026-09-17T19:19:47.551Z	2026-09-17T19:41:55.102Z
2026-09-17T16:41:54.881-03:00	DEV-1123	AU-210	39990	15	danado_por_uso	RECHAZO	Producto danado por uso	74000	21	40950.9	0.98	cliente9@example.cl	4	2026-09-17T19:19:47.615Z	2026-09-17T19:41:55.159Z
2026-09-17T16:41:54.887-03:00	DEV-1118	MW-150	79990	4	sin_abrir	REVISION	Monto sobre el umbral	74000	21	40950.9	1.95	cliente4@example.cl	5	2026-09-17T19:19:47.674Z	2026-09-17T19:41:55.218Z
2026-09-17T16:41:54.893-03:00	DEV-1120	AS-260	89990	2	con_fallas	REVISION	Monto sobre el umbral	74000	21	40950.9	2.2	cliente6@example.cl	6	2026-09-17T19:19:47.735Z	2026-09-17T19:41:55.276Z
2026-09-17T16:41:54.899-03:00	DEV-1121	LV-330	289990	9	sin_abrir	REVISION	Monto sobre el umbral	74000	21	40950.9	7.08	cliente7@example.cl	7	2026-09-17T19:19:47.804Z	2026-09-17T19:41:55.341Z
2026-09-17T16:41:54.906-03:00	DEV-1126	MW-150	79990	8	con_fallas	REVISION	Monto sobre el umbral	74000	21	40950.9	1.95	cliente12@example.cl	8	2026-09-17T19:19:47.869Z	2026-09-17T19:41:55.410Z
2026-09-17T16:41:54.913-03:00	DEV-C02	LV-330	289990	2	sin_abrir	REVISION	Monto sobre el umbral	74000	21	40950.9	7.08	control02@example.cl	9	2026-09-17T19:19:47.936Z	2026-09-17T19:41:55.493Z
2026-09-17T16:41:54.918-03:00	DEV-1117	HV-050	24990	12	con_fallas	REVISION	Producto con fallas	74000	21	40950.9	0.61	cliente3@example.cl	10	2026-09-17T19:19:48.002Z	2026-09-17T19:41:55.553Z
2026-09-17T16:41:54.924-03:00	DEV-1115	AU-210	19990	3	sin_abrir	APROBACION	Cumple condiciones de aprobacion	74000	21	40950.9	0.49	cliente1@example.cl	11	2026-09-17T19:19:48.056Z	2026-09-17T19:41:55.609Z
2026-09-17T16:41:54.931-03:00	DEV-1116	TE-330	69980	5	abierto_sin_uso	APROBACION	Cumple condiciones de aprobacion	74000	21	40950.9	1.71	cliente2@example.cl	12	2026-09-17T19:19:48.113Z	2026-09-17T19:41:55.667Z
2026-09-17T16:41:54.938-03:00	DEV-1122	PL-120	45990	2	sin_abrir	APROBACION	Cumple condiciones de aprobacion	74000	21	40950.9	1.12	cliente8@example.cl	13	2026-09-17T19:19:48.170Z	2026-09-17T19:41:55.737Z
2026-09-17T16:41:54.946-03:00	DEV-1124	HV-050	49980	1	abierto_sin_uso	APROBACION	Cumple condiciones de aprobacion	74000	21	40950.9	1.22	cliente10@example.cl	14	2026-09-17T19:19:48.225Z	2026-09-17T19:41:55.796Z
2026-09-17T16:41:54.955-03:00	DEV-C01	CB-010	9990	1	sin_abrir	APROBACION	Cumple condiciones de aprobacion	74000	21	40950.9	0.24	control01@example.cl	15	2026-09-17T19:19:48.285Z	2026-09-17T19:41:55.853Z

Figura 3: Registro final de las 15 solicitudes procesadas.

2.5. Resultados del lote base

Ruta	Cantidad	Monto total
APROBACION	5	\$195.930
REVISION	6	\$854.940
RECHAZO	4	\$200.950
DATOS_INVALIDOS	0	\$0
Total	15	\$1.251.820

Tabla 2: Resumen de resultados del workflow principal.

La tasa de aprobación automática es:

$$\frac{5}{15} \times 100 = 33,33\%.$$

Cuando una solicitud cumple más de una condición, se utiliza la primera que aparece en el orden definido. DEV-1119 tiene 25 días y estado **danado_por_uso**, por lo que se rechaza por plazo máximo excedido antes de revisar el estado del producto. DEV-C03 tiene 29 días y un monto de \$79.990, por lo que también se rechaza por plazo antes de llegar a la regla de revisión por monto. DEV-1120 supera el umbral y además tiene estado **con_fallas**; queda en revisión por monto, ya que esa condición aparece primero dentro de la ruta de revisión.

ID	Monto	Días	Estado	Ruta	Motivo
DEV-1115	\$19.990	3	sin_abrir	APROBACION	Cumple condiciones de aprobación
DEV-1116	\$69.980	5	abierto_sin_uso	APROBACION	Cumple condiciones de aprobación
DEV-1117	\$24.990	12	con_fallas	REVISION	Producto con fallas
DEV-1118	\$79.990	4	sin_abrir	REVISION	Monto sobre el umbral
DEV-1119	\$45.990	25	danado_por_uso	RECHAZO	Plazo máximo excedido
DEV-1120	\$89.990	2	con_fallas	REVISION	Monto sobre el umbral
DEV-1121	\$289.990	9	sin_abrir	REVISION	Monto sobre el umbral
DEV-1122	\$45.990	2	sin_abrir	APROBACION	Cumple condiciones de aprobación
DEV-1123	\$39.980	15	danado_por_uso	RECHAZO	Producto dañado por uso
DEV-1124	\$49.980	1	abierto_sin_uso	APROBACION	Cumple condiciones de aprobación
DEV-1125	\$34.990	22	sin_abrir	RECHAZO	Plazo máximo excedido
DEV-1126	\$79.990	8	con_fallas	REVISION	Monto sobre el umbral
DEV-C01	\$9.990	1	sin_abrir	APROBACION	Cumple condiciones de aprobación
DEV-C02	\$289.990	2	sin_abrir	REVISION	Monto sobre el umbral
DEV-C03	\$79.990	29	abierto_sin_uso	RECHAZO	Plazo máximo excedido

Tabla 3: Resultado individual de las 15 solicitudes del lote base.

3. Parte B — Flujo programado de resumen

3.1. Funcionamiento del workflow

El workflow de resumen está configurado con un *Schedule Trigger* diario a las 20:00. Primero lee todos los registros de la tabla `registro_devoluciones`, conserva únicamente los de la fecha actual y utiliza el nodo *Summarize* para agrupar por *ruta* y sumar *monto*. Después reúne los grupos en un solo ítem y calcula las métricas del día.

La secuencia utilizada es:

Schedule Trigger → Leer registro → Filtrar hoy → Summarize
→ Aggregate → Calcular métricas → Preparar mensaje → Gmail

La Figura 4 muestra una ejecución del flujo. Se leen los registros del día, se generan los grupos por ruta y luego se prepara un único ítem para el correo.

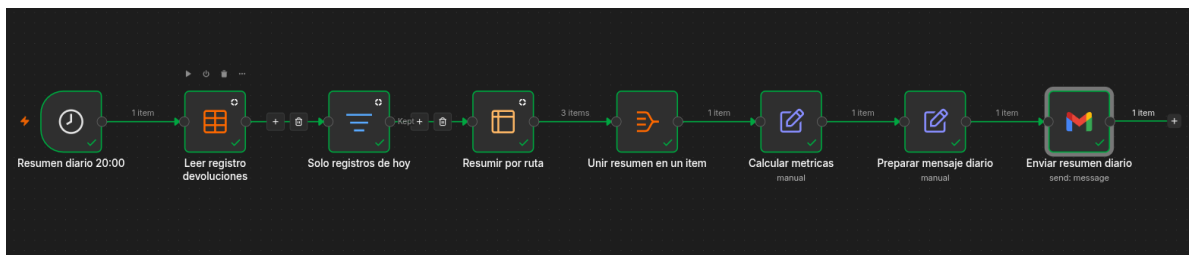


Figura 4: Ejecución del workflow de resumen diario.

3.2. Contenido del resumen

El correo incluye solicitudes por ruta, monto acumulado por ruta, monto total del día y tasa de aprobación automática. La tasa se calcula como $100 \times \text{aprobaciones} / \text{total}$ cuando existen solicitudes y se fija en cero si el total del día es cero.

En la ejecución utilizada como evidencia se obtuvo:

Ruta	Solicitudes	Monto
Aprobación automática	5	\$195.930
Revisión manual	6	\$854.940
Rechazo	4	\$200.950
Datos inválidos	0	\$0

Tabla 4: Métricas del resumen diario.

El asunto del correo se configuró como **Resumen diario | fecha_resumen**. El cuerpo se envía en HTML y presenta las cifras por ruta junto con el resumen general. La Figura 5 muestra el correo recibido durante la prueba.

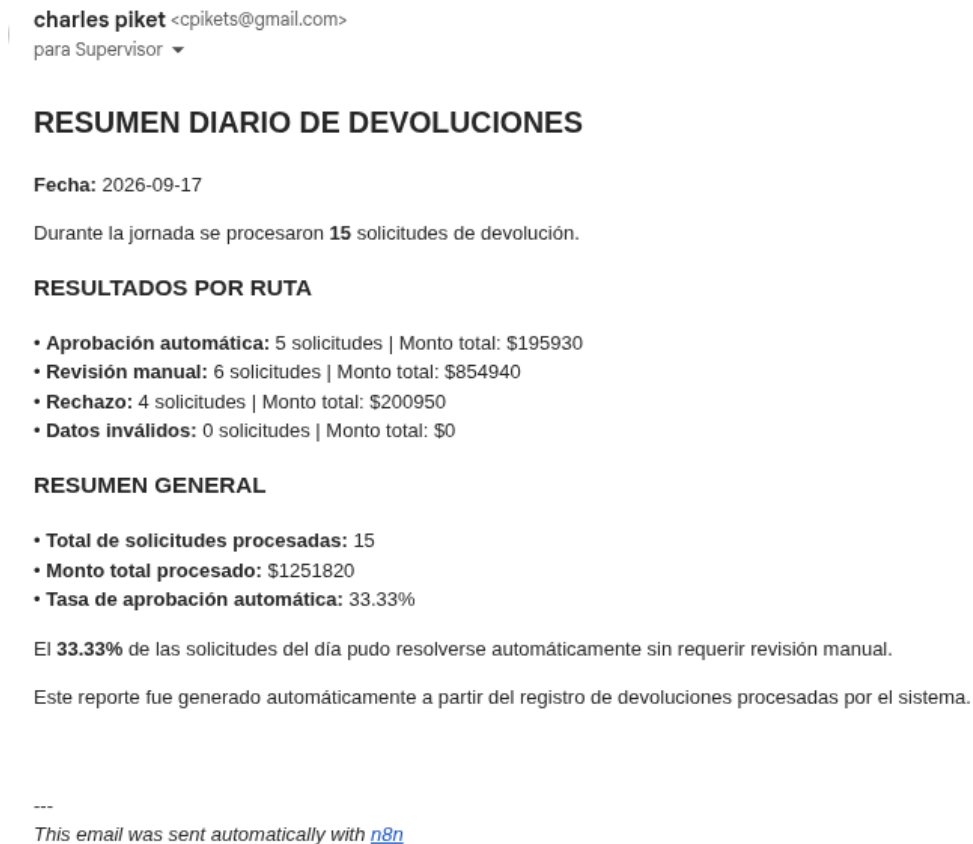


Figura 5: Correo generado por el workflow de resumen diario.

3.3. Día sin movimientos

El workflow contempla el caso en que no existan registros para la fecha de ejecución. Las búsquedas de cada ruta utilizan cero como valor por defecto, el total diario queda en cero, la tasa de aprobación se fija en 0 % y el mensaje informa que no hubo movimientos durante la jornada.

4. Parte C — Errores, bordes y límites

4.1. Error Workflow y configuración de errores

Se creó un Error Workflow con la secuencia *Error Trigger* → *Preparar aviso de error* → Gmail. El nodo intermedio extrae el nombre del workflow, identificador de ejecución, último nodo ejecutado, mensaje de

error y fecha, y construye el correo de aviso. El workflow principal tiene asociado este Error Workflow desde su configuración.

Un Error Workflow permite reaccionar a una ejecución que termina con error sin depender de que una persona esté observando n8n. Esto es especialmente útil para un flujo programado: si se ejecuta fuera del horario de supervisión y falla, el error puede quedar registrado pero pasar inadvertido si no existe una notificación externa.

n8n ofrece tres comportamientos de *On Error*: detener el workflow; continuar usando la salida normal; o continuar usando una salida específica de error. En la configuración revisada se utiliza lo siguiente:

Nodo(s)	On Error	Razón
Webhook; Normalizar entrada; Unir invalidos; Unir rechazo; Unir revision; Switch1; Marcar datos invalidos1; Marcar rechazo1; Marcar revision1; Marcar aprobacion1; Decision consolidada2	Stop Workflow (predeterminado)	Una falla de recepción, conversión o clasificación impide asegurar una decisión correcta. El caso de monto = "veinte mil" confirma este comportamiento en Switch1.
Consultar UF1	Stop Workflow (predeterminado) + Retry On Fail	Antes de detener la ejecución realiza hasta tres intentos, separados por 1000 ms, para tolerar fallas transitorias de la API.
Unir datos y UF; Agregar datos UF1; Preparar registro; Crear tabla si no existe; Esperar tabla y continuar; Registrar solicitud; Respond to Webhook	Stop Workflow (predeterminado)	Una falla en la combinación, preparación, persistencia o respuesta impide cerrar correctamente la solicitud y debe dejar la ejecución como fallida.
Notificar cliente1	Continue using regular output	El correo al cliente es una rama paralela. Una falla de notificación no debe impedir que la otra rama registre la solicitud y responda al Webhook.
Continue using error output	No utilizado en el workflow principal	No se creó una salida de error separada por nodo; los errores que detienen la ejecución quedan centralizados mediante el Error Workflow.

Tabla 5: Configuración de On Error del workflow principal. Los nodos sin un valor explícito en el archivo exportado mantienen el comportamiento predeterminado.

En el workflow de resumen diario y en el Error Workflow tampoco aparecen cambios explícitos de *On Error* en los archivos exportados; por ello mantienen el comportamiento predeterminado de detener la ejecución ante una falla de nodo.

Durante las pruebas también apareció un problema de autenticación de Gmail con el mensaje *Unable to sign without access token*. La credencial OAuth fue reconectada y el envío volvió a funcionar. La prueba manual del Error Workflow se conserva en el Anexo A, ya que sirve como evidencia de la construcción del mecanismo, pero no reemplaza la ejecución fallida solicitada para los casos inválidos.

4.2. Tres entradas inválidas

Se probaron tres entradas inválidas distintas. La carga enviada, el resultado esperado y el observado se resumen en la Tabla 6. Las capturas completas se dejaron en el Anexo A para no interrumpir el desarrollo principal.

Entrada	Comportamiento esperado	Comportamiento observado
<code>id_solicitud</code> vacío	Clasificar como DATOS_INVALIDOS antes de evaluar rechazo o revisión.	Se activó la salida INVALIDO_CAMPO y la solicitud continuó por DATOS_INVALIDOS con motivo <code>id_solicitud invalida</code> .
<code>monto = 0</code>	Clasificar como DATOS_INVALIDOS por la regla <code>monto ≤ 0</code> .	Se activó INVALIDO_MONTO. Aunque el estado era <code>con_fallas</code> , la precedencia mantuvo la ruta DATOS_INVALIDOS y el motivo <code>Monto invalido</code> .
<code>monto = "veinte mil"</code>	La conversión numérica no puede completarse; al menos un nodo debe fallar.	La ejecución falló en Switch1 al evaluar una condición que utiliza <code>toNumber()</code> . Este caso muestra que el flujo no valida previamente que el monto sea numérico.

Tabla 6: Entradas inválidas probadas en la Parte C.

El tercer caso cumple la exigencia de provocar una falla real de un nodo. La diferencia respecto de los dos primeros es que el monto no numérico no pertenece a una ruta de negocio controlada: produce un error antes de que el flujo pueda asignar una ruta.

4.3. Casos de frontera

Se construyeron dos solicitudes adicionales para probar exactamente los límites U y D .

En BORDE-U se utilizó `monto = 74000`, `dias_desde_compra = 5` y estado `sin_abrir`. El resultado observado fue APROBACION. La condición de revisión es `monto > 74000`, por lo que la igualdad no activa REVISION_MONTO.

En BORDE-D se utilizó `dias_desde_compra = 21`, `monto $29.000` y estado `sin_abrir`. El resultado observado también fue APROBACION. La condición de rechazo es `dias_desde_compra > 21`, por lo que la igualdad no activa RECHAZO_PLAZO.

Las entradas y la ejecución de aprobación se muestran en el Anexo A. Los dos resultados confirman el comportamiento inclusivo pedido en la regla 6 del enunciado.

4.4. Error de tipo intencional

El enunciado solicita cambiar temporalmente la condición de monto del Switch de *Number* a *String*, enviar DEV-C01 con monto "9990", mostrar la ruta obtenida, explicar el comportamiento y luego revertir el cambio. Esta prueba todavía no cuenta con evidencia de ejecución disponible en los archivos entregados para esta actualización, por lo que debe completarse antes de la entrega final. En la configuración normal, DEV-C01 se procesa como número mediante `toNumber()` y termina en APROBACION.

4.5. Casos que el flujo no puede resolver

El caso `monto = "veinte mil"` muestra una limitación de validación de tipo: el Switch intenta convertir el texto a número y la ejecución falla. Para resolverlo en una versión definitiva se debería validar primero que el campo contenga un valor numérico antes de aplicar las comparaciones de negocio.

También se probó una solicitud con `sku = ZZ-999`, manteniendo valores válidos en los demás campos. El resultado observado fue APROBACION. Esto ocurre porque el flujo no verifica la existencia del SKU contra el catálogo: mientras `monto`, `plazo` y `estado` sean válidos, el SKU no interviene en la clasificación. Para resolverlo se necesita consultar el catálogo antes del triage y derivar los SKU inexistentes a una ruta de datos inválidos o revisión. La carga utilizada se muestra en el Anexo A.

5. Trazabilidad y controles de entrega

Cada solicitud queda registrada con identificador, SKU, monto, días desde la compra, estado, ruta, motivo, *U* y *D*. También se guarda el valor de la UF utilizado, el monto equivalente en UF, la fecha de registro y el correo del cliente. El resumen diario utiliza esos mismos registros.

El repositorio público se encuentra indicado en la portada y en el resumen ejecutivo. Antes de la exportación definitiva se deben revisar los tres archivos JSON y confirmar que no contengan contraseñas, tokens, valores Bearer ni datos fijados manualmente. En la exportación del workflow principal utilizada para esta revisión todavía existe contenido en `pinData`; por tanto, antes de entregar se debe hacer *unpin* de esos datos y volver a exportar el workflow. Los workflows de resumen diario y Error Workflow revisados sí presentan `pinData` vacío.

Además del informe, la entrega debe contener el workflow principal de triage, el workflow emisor, el workflow programado de resumen y las capturas de al menos dos ejecuciones exitosas y una fallida con el panel de ejecución visible. Las evidencias complementarias utilizadas en este informe se organizan en el anexo siguiente.

Conclusión

El workflow principal procesó correctamente las 15 solicitudes del lote base y mantuvo la trazabilidad desde la entrada hasta el registro final. El resultado fue de 5 aprobaciones, 6 revisiones y 4 rechazos. La consulta de UF, el registro con *Upsert*, la notificación al cliente y la respuesta del Webhook quedaron integrados al flujo.

Las pruebas adicionales confirmaron la precedencia de las reglas para datos inválidos y el comportamiento de los límites *U* y *D*. También permitieron identificar dos límites concretos del MVP: un monto no numérico puede detener la ejecución y un SKU inexistente puede ser tratado como válido. El flujo programado entrega el resumen diario solicitado y el Error Workflow proporciona un mecanismo de aviso ante fallas.

La implementación sirve como MVP y como base para continuar el desarrollo, pero antes de considerarla definitiva se deben cerrar la prueba intencional de tipo solicitada en la Parte C, añadir validaciones de entrada y catálogo, y completar los controles finales de exportación y evidencia.

A. Evidencias complementarias

Este anexo reúne capturas necesarias para verificar configuraciones y casos de prueba sin recargar el desarrollo principal.

A.1. Configuración del Switch

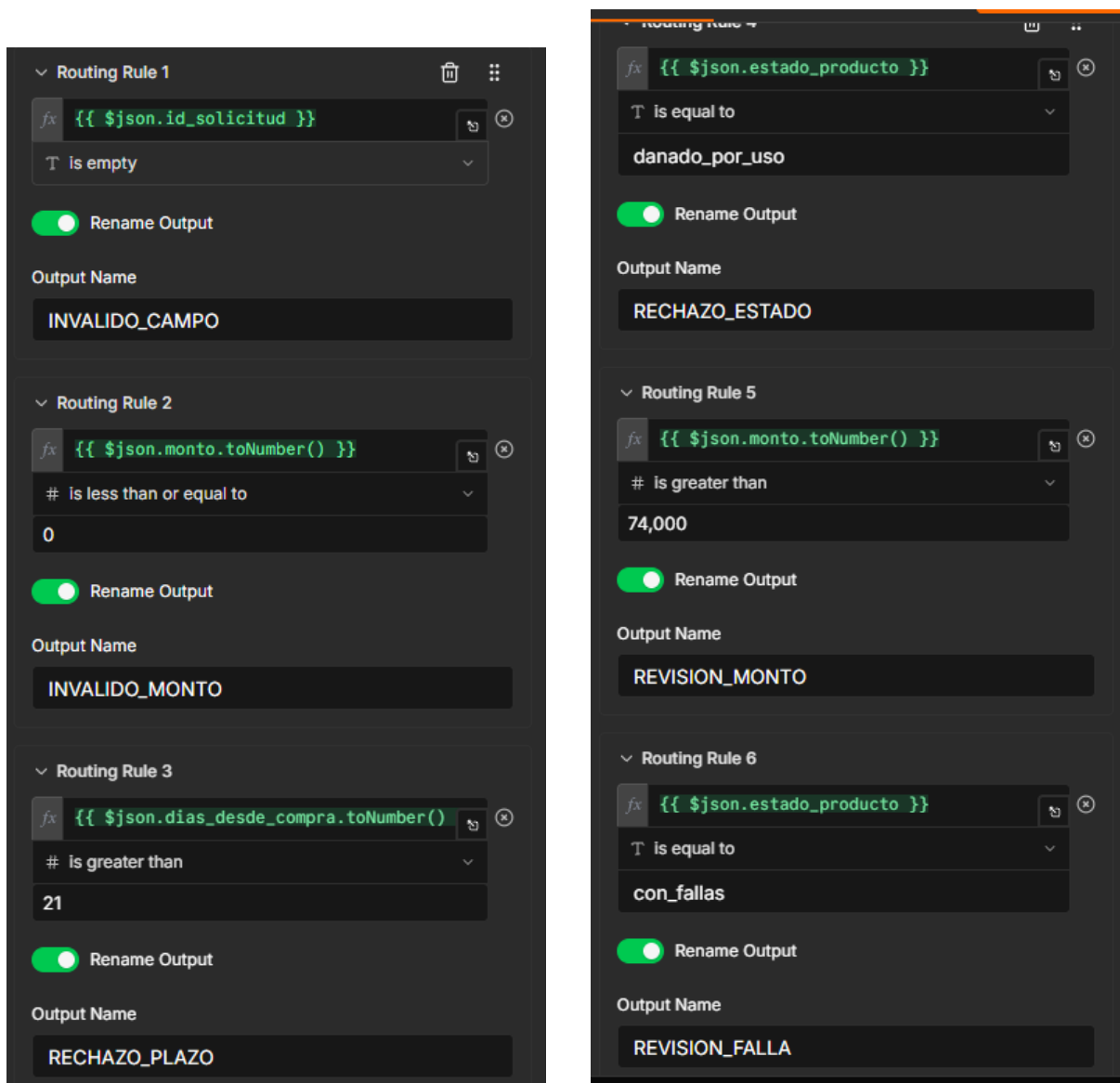


Figura 6: Reglas 1 a 6 configuradas en el nodo Switch.

A.2. Caso inválido: identificador vacío

1 item						
id_solicitud	sku	unidades	monto	dias_desde_compra	estado_producto	email_cliente
empty	HV-050	1	29000	12	con_fallas	cliente3@example.cl

Figura 7: Carga de prueba con id_solicitud vacío.

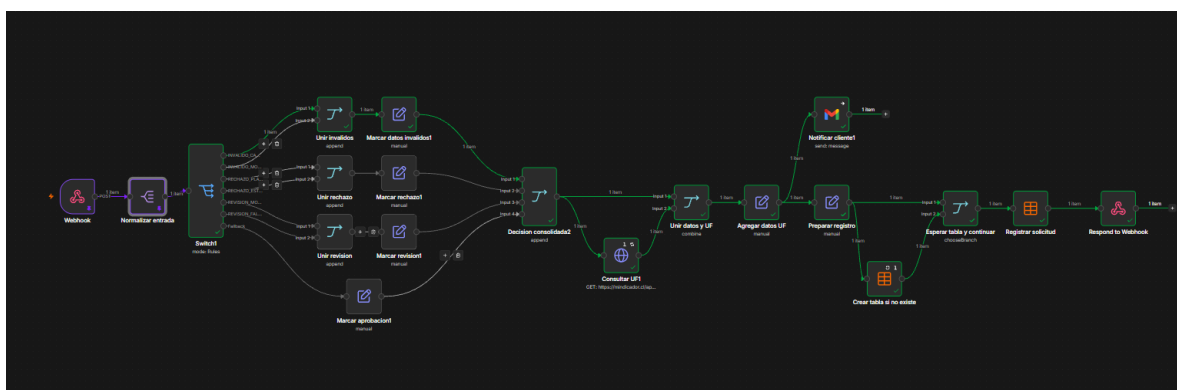


Figura 8: Ejecución del caso con identificador vacío por la ruta de datos inválidos.

A.3. Caso inválido: monto igual a cero

1 item						
id_solicitud	sku	unidades	monto	dias_desde_compra	estado_producto	email_cliente
DEV-1116	HV-050	1	0	12	con_fallas	cliente3@example.cl

Figura 9: Carga de prueba con monto = 0.

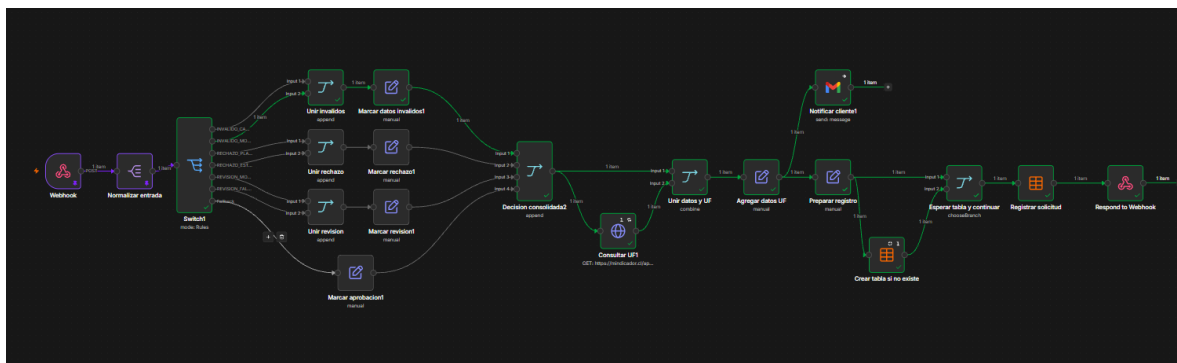


Figura 10: Ejecución del caso monto = 0 por la ruta de datos inválidos.

charles piket <cpikets@gmail.com>
para cliente3 ▾

Su solicitud DEV-1116 fue procesada. Resultado: DATOS_INVALIDOS Motivo: Monto invalido Monto: \$0 Monto aproximado en UF: 0 Saludos, AndesHogar SpA

This email was sent automatically with [n8n](#)

Figura 11: Notificación generada para el caso `monto = 0`.

A.4. Caso inválido: monto no numérico

1 item

id_solicitud	sku	unidades	monto	dias_desde_compra	estado_producto	email_cliente
DEV-1116	HV-050	1	veinte mil	12	con_fallas	cliente3@example.cl

Figura 12: Carga de prueba con `monto = “veinte mil”`.

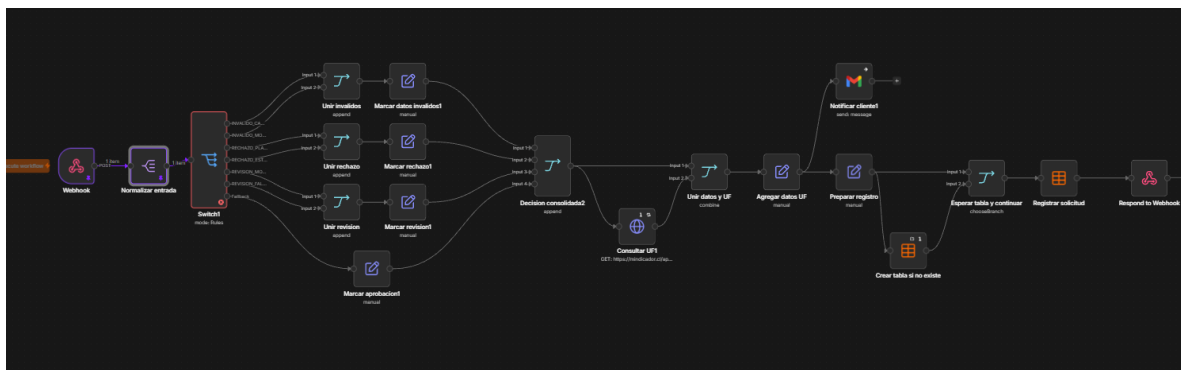


Figura 13: Ejecución fallida en Switch1 al procesar el monto no numérico.

A.5. Prueba manual del Error Workflow

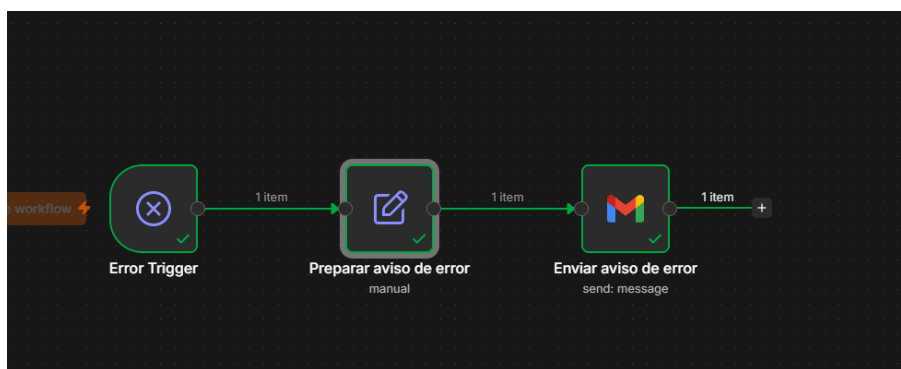


Figura 14: Ejecución manual del Error Workflow: Error Trigger, preparación del aviso y Gmail.

charles piket <cpikets@gmail.com>
para mí ▾

Se detectó un error en el workflow: Example Workflow Nodo: Node With Error Error: Example Error Message Execution ID: 231 Fecha: 2026-09-17T16:55:28.195-03:00

This email was sent automatically with [n8n](#)

Figura 15: Correo de prueba generado manualmente por el Error Workflow.

A.6. Casos de frontera

1 item						
id_solicitud	sku	unidades	monto	dias_desde_compra	estado_producto	email_cliente
BORDE-U	HV-050	1	74000	5	sin_abrir	cliente3@example.cl

Figura 16: Entrada del caso frontera $monto = U = 74000$.

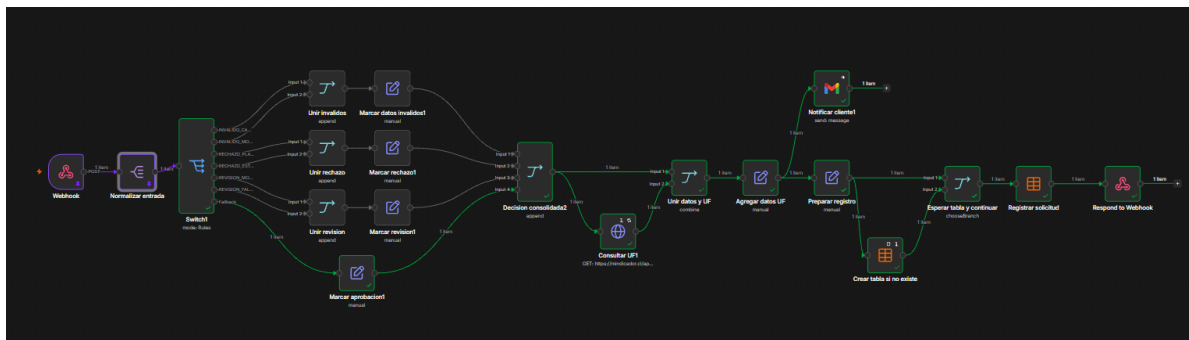


Figura 17: Ejecución por la rama de aprobación para el caso frontera.

1 item						
id_solicitud	sku	unidades	monto	dias_desde_compra	estado_producto	email_cliente
BORDE-D	HV-050	1	29000	21	sin_abrir	cliente3@example.cl

Figura 18: Entrada del caso frontera $dias_desde_compra = D = 21$.

A.7. SKU inexistente

1 item						
id_solicitud	sku	unidades	monto	dias_desde_compra	estado_producto	email_cliente
DEV-1116	ZZ-999	2	69980	5	sin_abrir	cliente2@example.cl

Figura 19: Entrada con $sku = ZZ-999$; el flujo la clasifica según monto, plazo y estado porque no valida catálogo.